

1. Bubble Sort - Sort Student Marks

Problem Statement:

A school maintains student marks in an array. Implement **Bubble Sort** to sort the student marks in **ascending order**.

Hint:

- Traverse through the array multiple times.
- Compare adjacent elements and swap if needed.
- Repeat the process until no swaps are required.

```
package BubbleSort;

public class BubbleSorting {
    public static void studentMarks(int arr[]){
        int n = arr.length;
        for(int i = 0; i < n; i++){
            for(int j = 0; j < n-1; j++){
                if(arr[j] > arr[j+1]){
                    int temp = arr[j];
                    arr[j] = arr[j+1];
                    arr[j+1] = temp;
                }
            }
        }
    }

    public static void main(String[] args) {
        int arr[] = {60,40,70,64,84,79};
        System.out.println("Array Before Sort");
        for(int i = 0; i < arr.length; i++){
            System.out.print(arr[i] + " ");
        }
        System.out.println();
        studentMarks(arr);

        System.out.println("Array After Sort");
        for(int i = 0; i < arr.length; i++){
            System.out.print(arr[i] + " ");
        }
    }
}
```

2. Insertion Sort - Sort Employee IDs

Problem Statement:

A company stores **employee IDs** in an unsorted array. Implement **Insertion Sort** to sort the employee IDs in **ascending order**.

Hint:

- Divide the array into sorted and unsorted parts.
- Pick an element from the unsorted part and insert it into its correct position in the sorted part.
- Repeat for all elements.

```
package InsertionSort;

public class InsertionSorting {
    public static void sortEmployeeId(int arr[]){
        int temp;
        int n = arr.length;
        for(int i = 0; i < n - 1; i++){
            for(int j = i + 1; j > 0; j--){
                if(arr[j] < arr[j-1]){
                    temp = arr[j];
                    arr[j] = arr[j-1];
                    arr[j-1] = temp;
                }
            }
        }
    }

    public static void main(String[] args){
        int arr[] = {101,110,104,103,113,111};
        System.out.println("Array Before Sort");
        for(int i = 0; i < arr.length;i++){
            System.out.print(arr[i] + " ");
        }
        System.out.println();
        sortEmployeeId(arr);
        System.out.println("Array After Sort");
        for(int i = 0; i < arr.length;i++){
            System.out.print(arr[i] + " ");
        }
    }
}
```

```
}  
}  
}
```

3. Merge Sort - Sort an Array of Book Prices

Problem Statement:

A bookstore maintains a list of book prices in an array. Implement **Merge Sort** to sort the prices in **ascending order**.

Hint:

- **Divide** the array into two halves recursively.
- **Sort** both halves individually.
- **Merge** the sorted halves by comparing elements.

```
package src.com.problems.sortings;  
  
import java.util.Scanner;  
  
public class MergeSort {  
  
    public static void mergeSort(int arr[], int start, int end){  
        if(start<end){  
            int mid=start+(end-start)/2;  
            mergeSort(arr, start, mid);  
            mergeSort(arr, mid+1, end);  
            merge(arr,start, end, mid);  
        }  
    }  
  
    public static void merge(int arr[], int start, int end, int mid){  
        int n1=mid-start+1;  
        int n2=end-mid;  
        int idx=start;
```

```

int la[]=new int[n1];
int ra[]=new int[n2];

for(int i=0; i<n1; i++){
    la[i]=arr[start+i];
    // System.out.println(la[i]+"lll ");
}

for(int i=0; i<n2; i++){
    ra[i]=arr[mid+i+1];
    // System.out.println(ra[i]+"rrr ");
}
// System.out.println("-----");
int itl=0,itr=0;
while(itl<n1 && itr<n2){
    if(la[itl]>ra[itr]){
        arr[idx]=ra[itr];
        itr++;
    }else{
        arr[idx]=la[itl];
        itl++;
    }
    // System.out.print(arr[idx]+" ");
    idx++;
}

while(itl<n1){
    arr[idx]=la[itl];
    // System.out.print(arr[idx]+" ");

    itl++;
    idx++;
}

while(itr<n2){
    arr[idx]=ra[itr];
    // System.out.print(arr[idx]+" ");

    itr++;
    idx++;
}

```

```
}

public static void printArr(int arr[]) {
    for (int ii : arr) {
        System.out.print(ii + " ");
    }
    System.out.println();
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int arr[] = new int[n];
    for (int i = 0; i < n; i++) {
        int ele = sc.nextInt();
        arr[i] = ele;
    }
    mergeSort(arr, 0, n-1);
    printArr(arr);
    sc.close();
}
}
```

4. Quick Sort - Sort Product Prices

Problem Statement:

An e-commerce company wants to display product prices in **ascending order**. Implement **Quick Sort** to sort the product prices.

Hint:

- Pick a **pivot** element (first, last, or random).
- **Partition** the array such that elements smaller than the pivot are on the left and larger ones are on the right.
- Recursively apply Quick Sort on left and right partitions.

```
package src.com.problems.sortings;

import java.util.Scanner;

public class QuickSort {
    public static int partition(int arr[], int start, int end) {
        int piv = arr[start];
        int i = start;
        int j = end;
        while (i < j) {
            while (arr[i] <= piv && i <= end - 1) {
                i++;
            }
            while (arr[j] > piv && j >= start + 1) {
                j--;
            }
        }
        if (i < j) {
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
        int temp = arr[start];
        arr[start] = arr[j];
        arr[j] = temp;
        return j;
    }

    private static void quickSort(int arr[], int start, int end) {
        if (start < end) {
            int par = partition(arr, start, end);
            quickSort(arr, start, par - 1);
            quickSort(arr, par + 1, end);
        }
    }

    public static void printArr(int arr[]) {
        for (int ii : arr) {
            System.out.print(ii + " ");
        }
    }
}
```

```

        System.out.println();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int arr[] = new int[n];
        for (int i = 0; i < n; i++) {
            int ele = sc.nextInt();
            arr[i] = ele;
        }
        quickSort(arr, 0, n - 1);
        printArr(arr);
        sc.close();
    }
}

```

5. Selection Sort - Sort Exam Scores

Problem Statement:

A university needs to sort students' **exam scores** in ascending order. Implement **Selection Sort** to achieve this.

Hint:

- Find the **minimum element** in the array.
- Swap it with the first unsorted element.
- Repeat the process for the remaining elements.

```

package SelectionSort;

public class SelectionSorting {
    public static void examScores(int arr[]){
        int n = arr.length;
        for(int i = 0; i < n; i++) {
            int minIndex = i;
            for(int j = i + 1; j < n ; j++){
                if(arr[j] < arr[minIndex]){

```

```

        minIndex = j;
    }
}
int temp = arr[minIndex];
arr[minIndex] = arr[i];
arr[i] = temp;
}
}
public static void main(String[] args) {
    int arr[] = {67, 70, 91, 73, 97, 61};
    System.out.println("Marks Before Sort");
    for (int i = 0; i < arr.length; i++) {
        System.out.print(arr[i] + " ");
    }
    System.out.println();
    examScores(arr);
    System.out.println("Marks After Sort");
    for (int i = 0; i < arr.length; i++) {
        System.out.print(arr[i] + " ");
    }
    System.out.println();
}
}

```

6. Heap Sort - Sort Job Applicants by Salary

Problem Statement:

A company receives job applications with different **expected salary demands**. Implement **Heap Sort** to sort these salary demands in **ascending order**.

Hint:

- Build a **Max Heap** from the array.
- Extract the largest element (root) and place it at the end.
- Reheapify the remaining elements and repeat until sorted.

```

package src.com.problems.sortings;

import java.util.Scanner;

```



```
public class HeapSort {
    public static void heapSort(int arr[], int idx, int n){
        int left=idx*2+1;
        int right=idx*2+2;
        int li=idx;
        if(left<n && arr[left]>arr[li]){
            li=left;
        }

        if(right<n && arr[right]>arr[li]){
            li=right;
        }

        if(li!=idx){
            int temp=arr[li];
            arr[li]=arr[idx];
            arr[idx]=temp;
            heapSort(arr, li, n);
        }
    }

    public static void createHeap(int arr[], int n){
        int start=(n/2)-1;
        for(int i=start; i>=0; i--){
            heapSort(arr,i,n);
        }
        for(int i=n-1; i>=0; i--){
            int temp=arr[i];
            arr[i]=arr[0];
            arr[0]=temp;
            heapSort(arr, 0, i);
        }
    }

    public static void printArr(int arr[]){
        System.out.println("Sorted Heap for given array : ");
        for(int ii : arr){
            System.out.print(ii+" ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
```

```
Scanner sc = new Scanner(System.in);
int n=sc.nextInt();
int arr[]=new int[n];
for(int i=0; i<n ;i++){
    int ele=sc.nextInt();
    arr[i]=ele;
}
createHeap(arr,n);
printArr(arr);
sc.close();
}
}
```

7. Counting Sort - Sort Student Ages

Problem Statement:

A school collects students' **ages** (ranging from 10 to 18) and wants them sorted. Implement **Counting Sort** for this task.

Hint:

- Create a **count array** to store the frequency of each age.
- Compute cumulative frequencies to determine positions.
- Place elements in their correct positions in the output array.

```
package src.com.problems.sortings;

import java.util.Scanner;

public class CountingSort {
    public static void countingSort(int arr[]){
        int n=arr.length;
        int mx=arr[0];
        for(int ii : arr){
            if(mx<ii)mx=ii;
        }
        int cnt[]=new int[mx+1];
```

```
int temp[]=new int[n+1];
for(int i=0; i<n; i++){
    cnt[arr[i]]++;
}
for(int i=1; i<=mx; i++){
    cnt[i]+=cnt[i-1];
}
for(int i=n-1; i>=0; i--){
    temp[cnt[arr[i]]-1]=arr[i];
    cnt[arr[i]]--;
}
// Arrays.fill(arr, 0);
for(int i=0; i<n; i++){
    arr[i]=temp[i];
}

}

public static void printArr(int arr[]) {
    for (int ii : arr) {
        System.out.print(ii + " ");
    }
    System.out.println();
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int arr[] = new int[n];
    for (int i = 0; i < n; i++) {
        int ele = sc.nextInt();
        arr[i] = ele;
    }
    countingSort(arr);
    printArr(arr);
    sc.close();
}
}
```